练习3 (选做)：利用循环和分块提升计算性能

- **任务**：
  - 修改练习2代码，设计恰当循环顺序和数量进一步提升矩阵乘法性能。
  - 本练习为选做内容，根据具体实现及加速效果加分。
- **目标**：
  - 学会通过优化算法提高缓存命中率；
  - 学会根据缓存大小优化矩阵乘法分块大小提升性能。

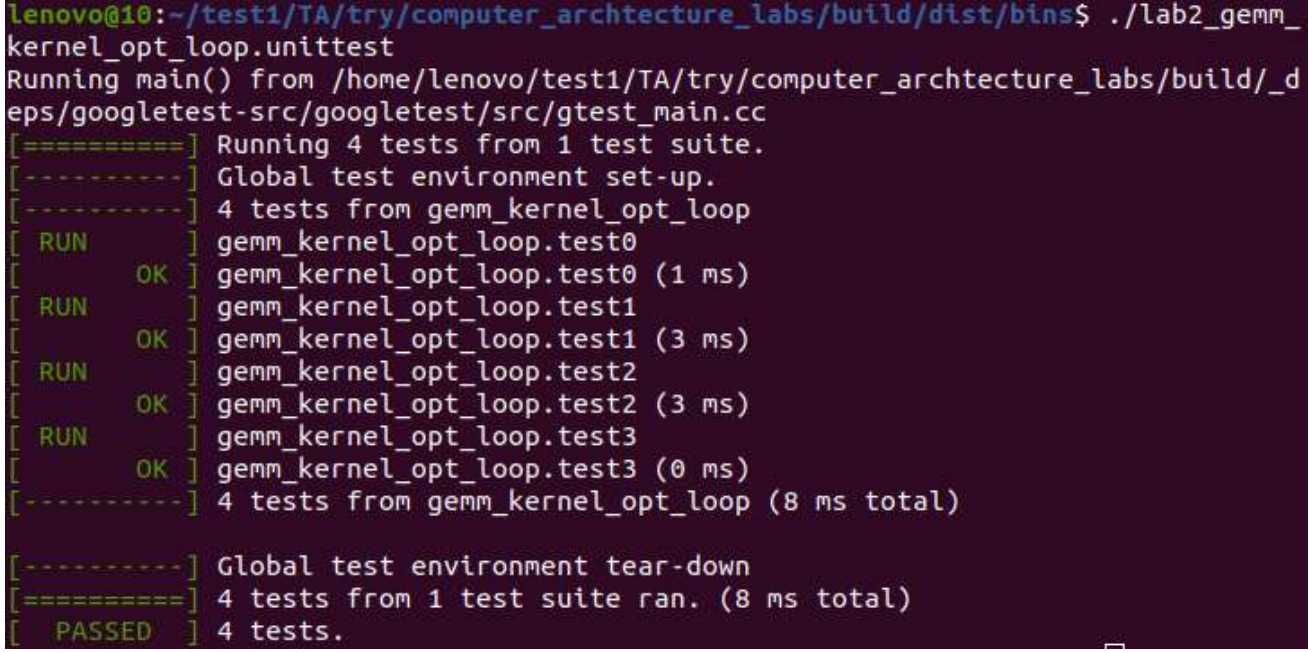
1. 根据练习1的分析结果，完善 `src/lab2/gemm_kernel_opt_loop.S` 中 `DO_GEMM` 过程（第55行）的矩阵计算逻辑



2. 验证kernel结果的正确性

在 `lab2/build` 目录下执行下列命令：

```
cmake -B . -S ../ && cmake --build ./ --target lab2_gemm_kernel_opt_loop.unittest
./dist/bins/lab2_gemm_kernel_opt_loop.unittest
```



3. 对比优化后的算法与基线的性能（测试用例根据自己的优化选择），根据对比结果分析优化方案的局限性

在`lab2/build`目录下执行下列命令：

```
cmake -B . -S ../ && cmake --build ./ --target lab2_gemm_opt_loop
./dist/bins/lab2_gemm_opt_loop 4 32768 4
```

```
lenovo@10:~/test1/TA/computer_archtecture_labs$ ./build/dist/bins/lab2_gemm_opt_loop 4 32768 4
--- Performance before loop optimization ---
GEMM performance info:
                M, K, N: 4, 32768, 4
                Ops: 0.00104858
                Total compute time(s): 0.121875
                Cost(s): 0.000609375
                Benchmark(Gflops): 1.72074
--- Performance for after loop optimization ---
GEMM performance info:
                M, K, N: 4, 32768, 4
                Ops: 0.00104858
                Total compute time(s): 0.111264
                Cost(s): 0.00055632
                Benchmark(Gflops): 1.88484
----------------------------
Performance difference(Gflops): 0.164103
```

尝试结合预取指令进一步优化算法，并对比优化前后的性能。